MeTTaDinC

A MeTTa sublanguage for AI systems that model and reason about their own workings and each other's

An expository introduction, 5 November 2026
Developed from the d-calculus and Hyperseed project

Summary

MeTTaDinC is a proposed sublanguage of MeTTa, the programming language of the Hyperon AGI framework. It gives an AI system a compact way to describe parts of itself, or of a peer, as small mathematical machines, each made of states, the operations that change them, and the questions other components ask about them. The system can then put precise questions to those descriptions. Does this compressed memory still answer what its users ask, today and after the next update? Does this revised goal still call for the same actions? Does this skill, translated into another robot's camera frame, still earn what it earned before? Judging from the logs, is the failure rate of this component below the rate we agreed to accept?

Each answer comes back as an atom that the system can store, reason over and pass on. The atom carries a status saying what kind of evidence stands behind it. A refutation comes with a counterexample that can be replayed, and a positive answer comes, wherever possible, with a proof object that a small independent checker can verify.

The mathematics behind the language is the distinction calculus, which was developed to model minds organized as hives of agents: systems that fork and merge, share memories and skills, and keep revising their own descriptions of themselves. MeTTaDinC is meant to make a tractable part of that mathematics usable inside running MeTTa-based systems, so that their self-description, self-reasoning and self-modification rest on checked statements rather than on guesses.

MeTTaDinC is complementary to Hyperseed. Hyperseed is an ontology for seed AGI, a vocabulary of agents, situations, contexts, evidence, distinctions, habits and commitments, and it links an AI system to the variety of real-world situations it has to handle. MeTTaDinC links the system's actual structures, dynamics and interactions to mathematics that can guide nontrivial reasoning about them. The two meet in the middle, and writing Hyperseed out explicitly in MeTTaDinC is a near-term goal.

This note explains what the language is, why it exists and what its programs look like. It draws its examples from the six documents in which MeTTaDinC has been specified and worked out on concrete problems. It closes with remarks on MeTTa sublanguages in general and a short guide to the six documents.

1. Why seed AGI needs a language for self-description

Self-modification is a series of decisions about distinctions

A seed AGI is a system meant to improve itself, and in the Hyperon and Omega setting that improvement happens in many small ways at once. An agent compresses its episodic memory, migrates a record schema, revises a goal, adopts a memory policy that a search process proposed, shares a skill with a peer that sees the world from another angle, or forks into copies that later have to be merged. Every one of these acts decides which differences between states the system will keep track of and which it will treat as irrelevant. A memory summary declares two histories equivalent when they produce the same summary, a goal declares two situations equivalent when it answers the same about both, and a translated skill declares a state of one agent equivalent to a state of another.

Mostly these decisions are made implicitly, by an engineer or by the agent itself, and their consequences show up later, when some downstream process needs a difference that was thrown away. The distinction calculus makes them explicit. In its terms an observer is a judgment of which things look alike, and the useful questions are whether that judgment survives the operations the system performs, how much is lost when it does not, and what the cheapest repair would be.

A concrete case

The memory layer of the Omega agent running on ProtoCosmo2, called WMTM (Working Medium-Term Memory), includes a goal engine that proposes follow-ups on open questions and pending items. The first live review of its suggestions rated only 18 percent of them useful or plausible. The report named the causes: stale items stayed open because old replies were missed, and paraphrases of one question were suggested again.

When the open-item store was modeled in MeTTaDinC, in Part I of the Omega memory guide, the first cause turned out to be a statement about distinctions with a short witness. Suppose the engine decides whether an item is closed by looking at what its recall window currently shows, with replies visible for two days. Then an item that was never answered and an item answered three days ago look the same, both open, and the engine proposes following up on a question that was settled. Worse, two items that both look closed today, one answered today and one answered two days ago, come apart after one more day: the older reply leaves the window and its item seems to reopen. No choice of window width repairs this, since a wider window only moves the day on which items reopen. Recording the closing as an event on the item, at the moment the reply arrives, passes every check. The same analysis also says how little the engine needs to remember. For its own purposes it needs only whether an item is open and, if it is, its age up to seven days, which makes nine classes of item in all.

None of this took deep mathematics. It took the right questions put to a precise description, and an answer that names the two items, the day and the query on which a design fails. That is the job MeTTaDinC is built for, and it becomes more pressing when the designer of the memory is the agent itself.

What such a language has to provide

Working through cases like this one led to a short list of requirements.

- It has to describe the system's own components: the states of a memory or goal store, the operations that change them, the queries other components ask, and the encodings that compress them.
- It has to answer exactly where exact answers are possible and say plainly when they are not. A claim established by exhaustive computation, a claim that rests on a solver's word, a claim estimated from logs and a question nobody could decide are different things, and a system that confuses them will act on unfounded beliefs about itself.
- Its answers have to be data that the system can reason over, store and send to others, together with the evidence behind them.
- It has to be cheap enough to run routinely. In practice that means small declared models, exact algorithms where they exist, solvers for statements about unbounded state, and statistics for what no declaration can settle.
- It has to live where the system lives, which for Hyperon-based systems means MeTTa.

2. The mathematics: the distinction calculus in brief

The distinction calculus is a body of mathematics for modeling minds that are hives of reflexively self-modifying agents, set out in working notes of September 2026 titled "The Distinction Calculus: d-Forms for Hives of Reflexively Self-Modifying Systems." The organizing idea is that much of what a mind does to its own descriptions can be read as operations on distinctions: creating them, merging them, carrying them from one agent to another, and measuring what they cost to keep. Four of its ideas run through MeTTaDinC.

Observers are tolerances. An observer on a set of states records, for every pair of states, how indistinguishable the two are to it, on a scale from 1 (it cannot tell them apart) to 0 (it separates them completely). A division of the states into classes is the yes-or-no special case. In this sense a memory summary, a goal, a sensor and a classifier are all observers.

Dynamics pull observers back. If one step of the system takes each state x to T(x), then the observer that looks one step ahead judges x and y alike when it judges T(x) and T(y) alike. An observer that never merges two states the next step would separate is stable under that step, and an encoding with this property can be kept up to date by itself, without going back to the states it compressed. Adding, again and again, the distinctions that the steps create produces the coarsest stable observer, which is the smallest memory that keeps answering correctly. For yes-or-no observers this is the classical algorithm of partition refinement. The graded version measures how far an unstable encoding is from being stable.

Brackets measure what an observer keeps. Given a probability distribution over states, the chance that two randomly drawn states look alike to an observer is that observer's weakness, an Occam-style quantity in the quantale form presented at AGI-25. Weakness falls when an observer learns distinctions and rises when the dynamics blur them, and the calculus keeps an account of both.

Hives join observers. A hive is a family of observers, such as agents, evaluators or memory views, together with correspondences between them. A hive is coherent when going around any cycle of correspondences returns each member to itself, up to a declared symmetry of what that member protects. When it does not, the failure shows up as a holonomy: a definite twist that a skill or a belief picks up on the way around the cycle. A library of skill translations between robots, a panel of evaluators and the forks of a single self-modifying agent are all hives in this sense.

In plain terms, the calculus treats every compression, comparison and translation inside a mind as a judgment of sameness, and it gives that judgment a precise shape that can be tested against what the mind will do next.

MeTTaDinC implements a finite, checkable part of this mathematics: tolerances on declared state sets, finite or described symbolically; the refinement and defect computations; simulation and transfer arrows between models; kernel hives and transfer hives; and the stochastic counterparts of these. Much of the calculus, such as its account of the branching time of agents that fork and merge, stays in the mathematics and guides the design rather than being executed.

3. What MeTTaDinC is

Modules, requests and results

A MeTTaDinC program is a module of declarations written in MeTTa syntax, followed by requests that ask questions about what the module declares.

```
(DC.Module name (Profile ...) declaration ...)
!(dc.run name request)
```

The declarations describe a small world. Carriers are sets of states. Maps are pure functions between carriers. A machine is a carrier together with actions, which change states, and observations, which are the queries that read them. A representation is a candidate encoding of a machine's states, such as a memory design. The language also has declarations for abstraction maps between machines, transfers between agents, hives, symmetries and transition kernels, and the companion documents propose further ones for summary statistics, logged data streams and the consumers of results. The answer to every request is a result atom with three parts:

```
(Result (Status ...)
  (Claim ...)
  (Evidence ...))
```

The specification describes two roles for the language. In the first it is a semantic workbench, in which researchers state finite thought experiments and get exact, reproducible answers. In the second it is an agent interface, in which representations, observers, methods, costs and evidence become data that the system's reasoning and control can use. The first role can be served well before the second matures, which is why the examples in the documents double as conformance tests for an implementation.

Statuses: what kind of evidence stands behind an answer

Every result carries one of seven statuses, and the documents follow one policy for assigning them.

Status	Meaning
`Checked`	Established by an exact computation, or by a proof object that the checker replayed
`Refuted`	A counterexample was found and replayed
`Conditional`	Follows if named assumptions hold, for example that a solver's report of unsatisfiability is correct when no proof of it was checked
`Estimated`	A statistical claim from logged data, reported with what it estimates, the population and selection rule of the data, and its error guarantee
`Unknown`	Unresolved, for example because a budget ran out or a certified interval straddles the bound; the result reports the work done
`Unsupported`	The running release does not implement this request for this kind of model
`InvalidInput`	The request is malformed, or it applies a result to an object the result is not about

Three rules make the statuses dependable. First, every result is bound to the exact module, objects, interface and scope it concerns, and using it for anything else is InvalidInput. Second, a result derived from Conditional results inherits all of their assumptions. Third, components that consume results declare which statuses they accept. A goal engine running inside the agent might accept results that rest on the cvc5 solver's word, while a protected goal held by the agent's governance frame accepts only Checked results; a promotion gate treats anything a consumer does not accept as missing evidence. One more point: every claim is about the declared model. A Checked claim about a model of a memory says something about the real memory only to the extent that the model describes it, and the documents check that link separately, by replaying the real system's logs through the model's maps.

Witnesses and proofs

A refutation always carries a witness: the states, the sequence of actions and the query on which the claimed sameness fails, small enough to follow by hand and to replay mechanically. A positive answer carries a certificate, a small piece of evidence that an independent checker can verify without redoing the search, wherever such a format exists. Examples are a splitting tree that justifies a partition, an inductive invariant, and a proof of unsatisfiability in the standard DRAT format, which a formally verified checker such as cake_lpr can replay. The worked examples ship 628 such proofs and replay all of them. Where no checkable proof is available yet, as with most solver answers about unbounded state, the result is Conditional on the solver, and a second solver's agreement is recorded beside it as evidence, without upgrading the status.

4. A first program

The open-item store of Section 1 makes a good first program. The module below is an excerpt from Part I of the Omega memory guide, shortened where marked with dots.

```
(DC.Module OpenItems
  (Profile Crisp)
  (Use NatArith Families)                              ; PROPOSED libraries
  (Carrier Age   (Of Nat) (Enum 0 1 2 3 4 5 6 7 8))      ; 8: more than seven days
  (Carrier Reply (Of Symbol) (Enum none d0 d1 d2 d3))    ; days since the reply; d3: 3+
  (Carrier Item  (Of (Tuple Nat Symbol)) (Product Age Reply))
  (Const start Item (Tuple 0 none))

  ;; how an item changes
  (Map later (From Reply) (To Reply) (Table (none none) (d0 d1) (d1 d2) (d2 d3) (d3 d3)))
  (Map tick (From Item) (To Item)
    (Cases ((Tuple $a $r) (Tuple (Min (Add $a 1) 8) (later $r)))))
  (Map on-reply (From Item) (To Item)
    (Cases ((Tuple $a $r) (Tuple $a (If (Eq $r none) d0 $r)))))

  ;; what its users ask
  (Map suggest (From Item) (To Bool) (Cases ((Tuple $a $r) (And (Eq $r none) (Le $a 7)))))
  (Map open    (From Item) (To Bool) (Cases ((Tuple $a $r) (Eq $r none))))

  (Machine ItemLife (States Item)
    (Observe suggest suggest) (Action tick tick) (Action reply on-reply))
  (Carrier Live (Of (Tuple Nat Symbol)) (Reach ItemLife (From start)))
  (Machine GoalEngine  (States Live) (Like ItemLife))              ; PROPOSED Like
  ...
  (Map in-window (From Reply) (To Bool)
    (Table (none False) (d0 True) (d1 True) (d2 True) (d3 False)))
  (Map window-view (From Item) (To (Tuple Nat Symbol))
    (Cases ((Tuple $a $r) (Tuple $a (If (in-window $r) closed open)))))
  (Map event-view (From Item) (To (Tuple Nat Symbol))
    (Cases ((Tuple $a $r) (Tuple $a (If (Eq $r none) open closed)))))
  ...
  (Representation WindowView (Of GoalEngine) (Encode window-view))
  (Representation EventView  (Of GoalEngine) (Encode event-view))
  ...
  (Derive EngineMemory     (MinimizeForQueries GoalEngine))
  (Derive CommitmentMemory (MinimizeForQueries Commitments))
  (Derive JointMemory      (MinimizeForQueries Both)))
```

Reading it from the top: an item is described by its age in days, with 8 standing for more than seven, and by the time since its closing reply, if any. Two actions change an item: a day passing (tick) and a reply arriving (reply). The goal engine asks one question, suggest: is the item open and at most seven days old? The agent's commitment record asks another, open. The carrier Live keeps only the states reachable from a new item. The two representations are the two record designs of Section 1. The window view infers closure from what a two-day recall window shows, and the event view records the closing as an event. The last three lines ask for the smallest memory that serves the engine, the commitment record, and both together; the elided lines declare the machines Commitments and Both, which ask open and both questions respectively.

The requests and their results:

```
!(dc.run OpenItems (CheckNow WindowView))
!(dc.run OpenItems (CheckUpdates WindowView))
!(dc.run OpenItems (CheckBehavioral EventView))
!(dc.run OpenItems EngineMemory)
!(dc.run OpenItems JointMemory)
!(dc.run OpenItems (CheckNow CapForCommitments))
```

Request	Status	Witness or value
`CheckNow WindowView`	Refuted	items (3, none) and (3, d3): same code, suggest answers true and false
`CheckUpdates WindowView`	Refuted	items (2, d0) and (2, d2), action tick: codes become (3, closed) and (3, open)
`CheckBehavioral EventView`	Checked	
`EngineMemory`	Checked	9 classes
`JointMemory`	Checked	10 classes
`CheckNow CapForCommitments`	Refuted	items (8, none) and (8, d0): open answers true and false

Three questions do most of the work in MeTTaDinC, and each can hold when the others fail. CheckNow asks whether an encoding answers the queries now, that is, whether two states with the same code always get the same answers. CheckUpdates asks whether the encoding can be kept up to date by itself: whether two states with the same code still have the same code after any one action. CheckBehavioral asks whether any sequence of future actions could reveal a difference the encoding has merged. The window view fails the first two, and its witnesses are exactly the two failure mechanisms described in Section 1. The event view passes all three.

The result of the first request, as the system receives it, is itself an atom:

```
(Result (Status Refuted)
  (Claim (Kind Now) (Representation WindowView) (Of GoalEngine))
  (Evidence (Witness (WordWitness (Pair (Tuple 3 none) (Tuple 3 d3))
              (SameCode (Tuple 3 open)) (Word (Do)) (Query suggest)
              (Left True) (Right False)))))
```

The memory requests carry a design lesson of their own. The engine alone needs nine classes of item, and the commitment record alone needs two (open or not). Together they need ten, one more than the engine, because the engine cannot tell an expired open item from a closed one, while to the commitment record an unanswered week-old question is still an open commitment. Applying the seven-day cap to the record itself (the representation CapForCommitments, elided above) makes the record forget that such an item is open, which is the refutation in the last row. The cap belongs to the engine's query, not to the stored record.

5. Key elements of the language

The table lists the forms used in the rest of this note. The specification defines many more, and its machine-readable registry is the authority on every form and field. Forms marked as proposed come from the companion documents, which define them through registry patches so that a static checker can verify every module against the specification together with the proposals.

Form	What it declares or asks
`(DC.Module name (Profile ...) ...)`	A module. The profile fixes the kind of dynamics (deterministic, stochastic or nondeterministic), the values (crisp or graded) and how carriers are represented (enumerated, symbolic or as snapshots of an atomspace).
`(Carrier name (Of type) ...)`	A set of states, given as an enumeration, a product, a subset, the states reachable from a start, or all values of a type (for symbolic carriers).
`(Map name (From A) (To B) body)`	A pure function, written as Cases, a Table or a Body in a restricted fragment of MeTTa that the checker can translate into solver obligations.
`(Machine name (States C) (Observe q f) (Action a g))`	The system or component being described: its states, its queries and its actions.
`(Representation name (Of M) (Encode f))`	A candidate encoding of machine M, such as a memory design or an index.
`(Derive name request)`	A named derivation, whose outputs later declarations can use.
`CheckNow, CheckUpdates, CheckBehavioral`	Is the representation adequate now, under each action, or for every future?
`Defects, Refine, MinimizeForQueries, InduceExact`	Measure what a representation loses, repair it, compute the smallest adequate memory, or derive the abstract machine that runs on the codes.
`AbstractionMap, CheckAbstractionMap, CheckSimulation, CheckSafety`	Maps between models that commute with their actions; one system simulating another; safety carried from a model to the system it simulates.
`Transfer, TransferHive, CheckTransferHive, Holonomy`	Moving skills between agents, and checking that a library of transfers is coherent around its cycles.
`Statistic, CheckStatistic (proposed)`	A summary that can be updated by itself, checked by a solver for every state of a store of any size.
`Stream, EstimateRate, Monitor, Detect (proposed)`	Logged data pinned by its digest, with its population and selection rule; anytime-valid estimates; monitors that detect change.
`Consumer (proposed)`	Who reads a result, which statuses it accepts, who may change it (the agent or its protected frame), and how its answers translate into decisions.
`CheckInterpretation (proposed)`	Does a changed goal call for the same decisions as the old one?

6. From self-description to self-modification: examples from the documents

The examples below are taken from the six documents. They are chosen to show the range of things a MeTTa-based system can say and check about itself with the language: what to remember, how to change a goal, how to keep statistics about an unbounded store, how to run a memory change through a promotion gate, how agents in a hive share skills, and how to audit or monitor systems too large to enumerate.

Memory: what an agent needs to remember

Part I of the Omega memory guide declares eight patterns of Omega's memory and goal machinery as MeTTaDinC models. Besides the open items of Section 4 they cover the context block that decides what the language model sees, the consolidation of evidence and of an anchor's cues, task checkpoints that survive a restart, goals as observers on memory and the sorting of goal changes, conflicts between goals and a prohibition, MetaMo's principles measured on a motivational dynamics, and a monitor that watches the goal engine on live data. In each, the components of the agent become machines and observers, and the memory designs become representations whose adequacy the checker decides.

The task checkpoint is a typical case. The requirements of restarting a task become the observers of a task machine, MinimizeForQueries returns the smallest checkpoint that serves them, and a field that a candidate checkpoint lacks appears as a witness. In the pattern's model the missing field is the read set of a task step: without it, a checkpoint that answers both restart questions today cannot be maintained once evidence is withdrawn, and the checker returns the two states, the withdrawal and the question on which the restart would go wrong.

Example 1 of the four worked examples takes the same questions to a belief memory with evidence counts and two consumers, a reasoner that reads a truth value and a controller that reads a three-way status. It finds that neither consumer has a useful exact compression, that the original decay rule makes every compact summary worthless, and that a gentler decay bounds the error of a twenty-one-code summary at 1/5. It also separates three meanings of "storing a belief in fewer bits" (an index, a cold store that is rehydrated on use, and a memory that runs on its codes), which carry three different claims and get three different answers.

Goals: changing a goal without changing what it asks for

Goal change is where self-modification touches what a system is for, and MeTTaDinC separates two questions that are easy to run together. The first is informational: which situations does each goal tell apart, and are the old answers recoverable from the new ones? The second is about meaning: does the new goal call for the same decisions as the old one? A decoder, declared by each goal's consumer and held by the agent's protected frame, says what a goal's answers ask the agent to do. Part I shows why both questions are needed with a deliberately extreme case, a goal replaced by its negation:

```
  ;; a negated goal: the same information as nag, the opposite answer on every record
  (Map anti-nag (From Record) (To Bool) (Body ($x) (Not (nag $x))))
  (Representation AntiAsNag (Of Nag) (Encode anti-nag))
  (Machine AntiNagGoal (States Record) (Observe anti anti-nag) (ActionsOf Reply))
  ...
  (Map follow-up-if-true (From Bool) (To Decision) (Table (True follow-up) (False wait)))
  (Map wait-if-true      (From Bool) (To Decision) (Table (True wait) (False follow-up)))
  ...
  (Consumer NagFollowUps      (Of Nag)           (Accepts Checked) (Custody Frame)
    (Decode follow-up-if-true))
  ...
  (Consumer AntiNagFollowUps  (Of AntiNagGoal)   (Accepts Checked) (Custody Frame)
    (Decode follow-up-if-true))
  (Consumer AntiNagWaits      (Of AntiNagGoal)   (Accepts Checked) (Custody Frame)
    (Decode wait-if-true)))

!(dc.run ItemGoals (CheckNow AntiAsNag))
!(dc.run ItemGoals (CheckInterpretation NagFollowUps AntiNagFollowUps))
!(dc.run ItemGoals (CheckInterpretation NagFollowUps AntiNagWaits))
```

The nag goal asks whether an open item is at most two days old and should be followed up. Its negation carries exactly the same information, since each answer determines the other, so the informational checks pass and the two goals have the same four-class memory. Read through decoders that both take a true answer as a call to follow up, however, the two goals ask for different decisions on every one of the forty records, and CheckInterpretation is Refuted at the first of them:

```
(Result (Status Refuted)
  (Claim (Kind Interpretation) (Old NagFollowUps) (New AntiNagFollowUps))
  (Evidence (Witness (RecordWitness (Record (Tuple open 0 ben normal))
              (OldAnswer True) (NewAnswer False)
              (OldDecision follow-up) (NewDecision wait)))))
```

With the opposite decoder, under which a true answer means wait, the negated goal is only a renaming, and the check is Checked. Part I sorts proposed goal changes into six kinds on this basis: renaming, refinement, coarsening, migration, failed migration and change of content. Adding "days left" to the nag goal's answer is a refinement at no cost in memory. Extending the follow-up to items three days old is a change of content, with the item aged three days as its witness. Storing days left instead of age is a migration whose abstraction map commutes with every action. The classification is meant as evidence for GOLEM-Iter's promotion gate, which must decide whether a proposed change to an agent endangers a protected goal. For a protected goal, a refinement counts as a semantic refinement in GOLEM-Iter's sense only with a Checked interpretation result whose decoders the frame holds, a migration needs its commuting map, and a change of content is normative succession, which goes through the external ceremony GOLEM-Iter reserves for constitutional changes.

Live memory: statistics that hold for every state and every store size

Finite models settle questions about one item at a time, over a few dozen states. A live store holds any number of items, whose ages are real numbers of days, and Part II of the guide shows how MeTTaDinC states the same memory decisions there as summary statistics. A statistic is an encoding together with an update rule for each action and a read rule for each query. If the encoding commutes with the actions and the reads answer the queries, a store can keep only the statistic and stay exact forever. The checks are posed to a solver for every state at once, over real-valued carriers:

```
  ;; the event statistic: the age and whether the item is closed
  (Map event-stat (From Item) (To (Tuple Rational Bool))
    (Cases ((Tuple $a $c $e) (Tuple $a $c))))
  ...
  (Statistic EventStat (Of ItemTime) (Encode event-stat)          ; PROPOSED Statistic
    (Step (wait $d) (event-wait $d) (For $d Days)) (Step reply event-reply)
    (Read suggest event-suggest) (Read open event-open))
  ...
  (Store Items (Keys ItemId) (Of ItemTime) (Global wait))         ; PROPOSED Store
  (Keyed EventIndex (Of Items) (Item EventStat))                  ; PROPOSED Keyed

!(dc.run LiveOpenItems (CheckStatistic EventStat (Method Symbolic)))
!(dc.run LiveOpenItems (CheckStatistic EventIndex (Method Lift)))
```

The first request sends a small set of obligations to the cvc5 solver; the second lifts the item-level result to a store of any number of items. Because no solver proof is checked yet, these results are Conditional on cvc5, with the agreement of the Z3 solver on independently written encodings recorded beside them. The same machinery checks a safety property of the

agent's arbitration rule through a simulation onto Part I's finite model, and it refuses to apply a certificate to the wrong object: a statistic checked for a different item machine, a simulation with a different target, or a stream of logged data whose content no longer matches its declared digest all give InvalidInput.

What no declaration can settle, such as how often the reply detector misses a reply, Part II measures from logged receipts. A stream declares its source, its digest and the population, selection rule and policy under which its rows were written, and EstimateRate returns an anytime-valid confidence bound that stays correct however often it is consulted:

```
  (Stream LiveSuggestions (Source "streams/stale_alt_seed1.json")
    (Digest "sha256:f0d4be8461a2904e728f7b0a8ad895385ccd56f340ec854d729936aaf6a48763")
    (Provenance "simulated: stale with probability 1/5, 400 suggestions, seed 1")
    (Population "suggestions of Part I's alternative engine")
    (Selection "one row per suggestion, written when it is made")
    (Policy "Part I's alternative engine, simulated, seed 1"))

!(dc.run LiveReceipts
   (EstimateRate (On LiveSuggestions) (Column "stale") (Equals 1) (Level (Rat 1 100))))
```

Part II proves that a one-sided bound of this kind remains valid when the rate varies from row to row, in which case it bounds the running average of the rows' conditional rates. This is what makes the bound usable on a live system, whose receipts are rarely draws from one fixed rate.

From journal to gate: one store, eight weeks

The WMTM worked example follows one store, the open items from which WMTM's goal engine proposes follow-ups, through eight simulated weeks of the engine's log-only phase. The data are simulated, generated from a fixed seed, with 1,357 items, 955 replies, a reply detector that misses some replies, spurious closing notes, paraphrased questions, and two record designs running side by side. Three short modules declare the store. A replay of the frozen journal through the module's maps agrees with the simulated designs on all 8,966 daily decisions, which ties the declared model to the system it describes.

On day 21 a parser update raises the detector's miss rate from 1.5 to 25 percent. Part II's mixture monitor never fires, because three healthy weeks count as evidence for the accepted rate. A Shiryaev-Roberts changepoint monitor, which the example adds as an option of Detect, fires 47 hours after the update; replies audited after the alarm confirm the regression, and a rollback and re-parse restore all 18 lost replies. At the promotion gate, claims from the reviewed receipts establish at level 0.01 that fewer than one reply in twenty is followed by a stale proposal, that fewer than one key in twenty is a duplicate, that fewer than one closing in twenty is false, and that the event design errs on fewer items than the window design. A random audit that checks the reviewers behind those receipts is still too small to establish its own two claims, and the gate promotes the engine on conditions it records with the claims.

The example ends by checking the next release, which adds a snooze action and extends the follow-up goal to three days, before any data from it exist. The deployed engine index becomes invalid input to the checker, since it has no update for the snooze, and the replacement index is the coarsest one the consumers allow. Every index can be rebuilt from the durable records, and the gate's claims carry over to the release, each by its own argument. The duplicate-rate claim carries over only because a snooze can be given only on a proposal, so it cannot keep a key

from being proposed for the first time. A release that let a person snooze an item before the engine ever proposed it would need new evidence.

Hives: sharing skills between agents

Example 3 of the worked examples is a hive in the sense of Section 2. Three warehouse robots see the same bay through cameras mounted in three different frames and share a library of navigation skills. A transfer from robot A to robot B reads each of B's states as a state of A and carries A's commands over to B, and a transfer hive declares the robots, the transfers between them, the cycles to check and the symmetries, called gauges, that each robot accepts as harmless:

```
  (Transfer AB      (Source A) (Target B) (StatePullback r270-cell) (MethodPushforward r90-dir))
  (Transfer BC      (Source B) (Target C) (StatePullback mv-cell)   (MethodPushforward mv-dir))
  (Transfer CA      (Source C) (Target A) (StatePullback r90-cell)  (MethodPushforward r270-dir))
  ...
  (TransferHive Strict (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CA) (Cycle A B C))
  ...
  (TransferHive Copied (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CA) (Cycle A B C)
    (Gauge A FlipUD-A) (Gauge B FlipUD-B) (Gauge C FlipUD-C))
  (TransferHive Transported (Member A) (Member B) (Member C)
    (Edge AB) (Edge BC) (Edge CA) (Cycle A B C)
    (Gauge A FlipUD-A) (Gauge B FlipLR-B) (Gauge C FlipLR-C))

!(dc.run SkillShare (CheckTransferHive Strict))
!(dc.run SkillShare (CheckTransferHive Copied))
!(dc.run SkillShare (CheckTransferHive Transported))
```

Each transfer is exact on its own edge, yet going around the cycle returns every cell as its mirror image, so a skill comes back earning at each cell what it earned at the mirrored cell. The strict hive is therefore Refuted, with the up-down mirror as its holonomy. Whether that is acceptable depends on the gauge declared at each robot. Copying the same mirror to every robot fails, because robot B's camera is turned a quarter turn, so A's up-down mirror is B's left-right mirror. A gauge transported correctly into each robot's frame passes. The example goes on to show a docking requirement breaking the symmetry and exposing a stale transfer, and a pallet in one bay making the transfers into it approximate. The same checks apply to any hive whose members exchange skills, beliefs or evaluations, including the forks of a single agent that has to merge them later.

Systems too large to enumerate, and environments that shift

Two further worked examples show the language beyond agent memory. Example 2 audits an administrative role-based access-control policy over four users and five roles, a state of forty bits that the checker handles symbolically rather than by enumeration. The audit finds two authoring mistakes, with two- and three-step witnesses. In the first, the rule for granting the approver role checks whether the user's auditor role is active in the current session, where it should check whether the role is assigned, so a user who is already a clerk can end up holding both roles after two assignments:

```
        (approver (And (holds $u clerk $x) (Not (active $u auditor $x))))
```

The corrected policy is proved safe with 80 small proofs, whose obligations are generated from the module source and whose proofs the verified checker replays. Restoring either mistake in

the source changes the verdict, and a targeted monitor for the policy turns out to need seventeen bits where a vendor proposed ten.

Example 4 models a market maker's quoting problem as a discounted decision process in four markets. In calm markets the desk can run on three risk buckets with no loss; toxic flow lowers every value by more than half and changes no decision; one-sided flow breaks both the compression and the policy. A news leak is invisible to a desk that only quotes wide, and each of two monitors is blind to one of the shifts. For an agent, the lesson carries over directly: a monitor is an observer too, and the language can say which changes in the world it cannot see.

7. MeTTaDinC and Hyperseed

Two kinds of connection

Hyperseed is an ontology for seed AGI. Its formal presentation treats contexts, evidence, distinctions, abstractions, descriptions and effort as mathematical structure, and the typed proof-plan language built on it organizes commonsense reasoning around a fixed vocabulary of object sorts, roles, relations and method families. That vocabulary was derived from a corpus of fifteen episodes chosen for their variety, among them the repair of a coffee maker, substituting one vessel for another, teaching fractions, a stressed colleague, a customer handoff, the revision of a governance rule and the erosion of a commitment. Hyperseed's job is to give an AI system words, and typed relations between them, for the situations the world presents.

MeTTaDinC's job lies on the other side. It connects the system's actual structures, dynamics and interactions (its memory stores, goal engines, transfers, forks and evaluators) to mathematics that can support nontrivial reasoning about them: which distinctions a component must keep, what a compression loses, whether a change preserves what a protected goal asks for, whether a library of transfers is coherent. A system needs both connections: a MeTTaDinC model of a memory store says precisely how the store behaves and nothing about what its items are about in the world, and Hyperseed's terms, applied to the system's own components without such a model, support only informal conclusions.

Where they meet

The two meet in concepts that both use, above all distinctness and interchangeability. In Hyperseed a context comes with two evaluators, one for how distinct two presented things are and one for how interchangeable they are, and the two need not be negations of each other. Two receipts can be distinct as receipts and interchangeable for the question currently being asked. Whether that interchangeability survives future use is exactly the kind of claim MeTTaDinC checks, for a declared consumer, and the result says which kind of adequacy was established (current, behavioral or trace) and whether equality is literal or only up to the consumer's observer. The specification gives a crosswalk from Hyperseed's constructions to MeTTaDinC representations and to the obligations that make them executable; a few of its rows:

Hyperseed construction	MeTTaDinC representation	What can be checked
Context	A record of presentation, evaluators, consumers, measures, costs and assumptions	Component types and compatibility; membership alone proves no adequacy
Observer, probe	A typed report map or an effect-scoped observation method	Frozen report semantics, acquisition law, availability convention
Interchangeable for a consumer	Equality up to the consumer's observer	Results marked as equality up to the observer, never read as identity
Abstraction	A representation with a code tolerance	Current, update and behavioral obligations under a stated claim kind
Weakness	A bracket in a named quantale	Declared order, product, measure and kernel
Pattern, simplicity	A description family with decoder, baseline, cost model and witness	Decoding and cost comparison; scoped search for the cheapest description
Forks, shared self-models	Fork and merge operations; kernel and transfer hives	Merge compatibility; coherence, holonomy and gauge

Hyperseed's truth values travel with it. Its p-bits, pairs of support and opposition with the four corners true, false, both and neither, are a library type in MeTTaDinC, with each operation named by the order it belongs to, and the language keeps the absence of a report apart from a report with neither support nor opposition. In the other direction, MeTTaDinC results enter Hyperseed's proof plans as warrants that keep their status, claim kind, scope and dependencies: a plan step that needs an abstraction cites a Checked induction result, and a step that relies on an estimate inherits the estimate's error guarantee.

The near-term goal

The registry already contains part of a Hyperseed library: p-bits and their operations, schemas, methods, description systems and pattern witnesses. The near-term goal is to write Hyperseed out explicitly in MeTTaDinC, its contexts, evaluators, abstractions, patterns, schemas, methods and claims, as declarations a checker can verify and an agent can use. That would make Hyperseed's vocabulary something a MeTTa-based system can apply to itself and to its peers with checked consequences, and it would give MeTTaDinC's models the world-facing vocabulary they lack on their own. The implementation guide schedules the corresponding library with observer discovery and the integration interfaces.

8. MeTTa sublanguages

Why MeTTa lends itself to sublanguages

From the start of MeTTa's design, one of the motivating ideas was that it should be a convenient medium for building domain-specific languages. Several features of the language support this.

A MeTTa program is made of atoms: symbols, variables, grounded atoms that wrap values and functions of a host language, and expressions, which are lists of atoms. Code and data share this one representation and live together in an atomspace, where they can be queried by pattern matching. Evaluation rewrites expressions by equations and by grounded functions, and a type system declares the types of atoms. A sublanguage can therefore be built from three ingredients without changing the host language: a vocabulary of expression forms with their types; an evaluator that gives the forms meaning, written in MeTTa as rewrite rules or in the host language behind a grounded operation; and a way to return results as atoms.

For seed AGI the most useful consequence follows from the shared representation. A sublanguage's programs and results are atoms in the same space as everything else the system knows, so the system that runs them can also read, generate, compare and change them. An agent can write a MeTTaDinC declaration of its own memory, ask whether a proposed compression is adequate, store the refutation it gets back, and reason about that refutation with PLN, all in one medium. For a sublanguage meant for self-description, that reflexivity is the main requirement.

How MeTTaDinC uses MeTTa

MeTTaDinC gives its forms meaning through a core written in the host language rather than through rewrite rules. A module is held as data, so the host evaluator does not rewrite its declarations; it is checked against a registry of forms, compiled to a typed intermediate representation, and executed by a core that calls exact algorithms, decision diagrams, SAT and SMT solvers and proof checkers. The maps inside a module are written in a restricted, pure fragment of MeTTa that the core can both evaluate and translate into solver obligations. The call !(dc.run Module request) returns a result atom to the surrounding MeTTa program. The semantic core does not depend on any particular evaluator's rewrite order, and the first host is Cetta, the direct C runtime for MeTTa; the implementation guide extends Cetta at a pinned revision rather than building a separate system.

Two design lessons from the work so far apply to MeTTa sublanguages generally. First, one machine-readable registry is the single source of truth for every form, field, argument and status: the specification's grammar appendix is generated from it, the static checker reads it, the implementation's frontend is to be generated from it, and a proposed form enters as a registry patch that the checker can test. Second, a sublanguage should state what its answers rest on. A small trusted core, results bound to the exact objects they concern, and one status policy that every component follows make it safe for one part of a system to act on another part's answers.

Sublanguages working together

MeTTaDinC is one of several sublanguages now taking shape. MeTTaProN is a probabilistic and neural sublanguage of MeTTa for writing probabilistic programs and neural models; one planned use expresses sampling-based control of PLN inference, with swarms of explorers searching between the forward and backward frontiers of a chain of reasoning. The two languages connect through a shared kernel representation rather than parallel syntax. A MeTTaDinC module names a pinned MeTTaProN model as a transition kernel:

```
(TransitionKernel Step (From X) (To X) (Import MeTTaProN step-model))
```

Exactness travels with the kernel. An exact kernel supports exact MeTTaDinC claims; an estimated or sampled kernel supports Estimated claims, or exact claims about the kernel that become Conditional on the assumption, declared with the import, that the kernel describes the system it was fitted to; a neural kernel from a frozen checkpoint enters with interval masses and its checkpoint identity. The same pattern links MeTTaDinC to its other neighbours. OmegaPLN's finite core with exact rational semantics is to serve as a differential oracle once its interpreter is pinned; the omega-PLN reasoner consumes MeTTaDinC results through an adapter in which exact results become exact channels and an estimate becomes evidence only through a conversion that keeps its estimand and its assumptions; MORK and PathMap supply carriers defined by queries over pinned snapshots of an atomspace; and Hyperseed's proof plans consume MeTTaDinC claims as warrants.

What these sublanguages begin to show is what the original idea of MeTTa as a medium for DSLs looks like in practice: a family of small languages, each with a narrow job and an explicit account of what its answers mean, sharing one representation, one atomspace and typed interfaces between them. The system using them can move from a probabilistic model in one language to a checked claim about it in another, and on to an inference that uses the claim, without leaving the medium in which it represents itself.

9. Where things stand

No MeTTaDinC runtime exists yet. Every result in the documents was computed from the module text by a reference evaluator written in Python, which is a test aid and not an implementation, and was computed again, independently, by Python models of the same systems; an oracle in each validation bundle compares the two and checks every number the text quotes. The specification's oracle runs 157 checks, and the oracles of the worked examples and of the three Omega documents run 305, 195, 94 and 99, all passing. Every module passes the static checker against the specification's registry together with its proposals.

The implementation guide hands the work to coding agents in eight milestones. It starts with strict boundaries for results, so that every result is bound to the objects it is about and every status means one thing. It then moves through a frontend generated from the registry, the graded deterministic core, symbolic carriers with proof-logged obligations, stochastic dynamics and estimates, nondeterminism and simulation, and the composition kernel with both kinds of hive, ending with Hyperseed, observer discovery and the integration interfaces. The first release is deliberately narrow: it must pass a suite of negative tests, deliver the 45 claims of its capability manifest, and answer Unsupported for the other 78.

For Omega the path runs through the documents' patterns. The memory layer's components become declared machines, a checker runs requests in the background and stores the results under the digest of the interface they concern, components consult the results through receipts, GOLEM-Iter's promotion gate consumes them as evidence, and the agent reaches them through a small set of functions. If that works, an Omega agent proposing a change to its own memory or goals will be able to say, in a form its peers and its gate can check, what the change preserves, what it gives up, and what evidence stands behind each claim, and a hive of such

agents will be able to do the same for the skills and beliefs its members exchange. That is the role we have in mind for MeTTaDinC in a seed AGI: one modest, checkable layer of self-understanding, built so that it can grow with the system it describes.

10. The documents

The six documents below make up the MeTTaDinC series as it stands, each available as a PDF with its LaTeX source. The specification comes with the validation package that defines the language's interfaces, and each of the last four documents comes with a validation bundle: the module files, the registry patches for any proposed forms, Python models, the reference evaluator and an oracle with its results.

MeTTaDinC: An Executable Language for Observer-Relative Abstraction, Evidence, and Composition (specification draft, 86 pages). The normative definition of the language. It sets out the trust model (claims about declared models, the seven statuses, what is trusted, how exact, estimated and conditional results differ), the semantic foundations in the distinction calculus, the graded core with its obligations, defects and certificates, the concrete syntax and evaluation, stochastic and nondeterministic profiles, graded kernels and approximation contracts, the composition kernel with transfers and hives, observer discovery, Hyperseed as an executable ontology, six application laboratories, integration with the rest of the Hyperon stack, and the validation package: the machine-readable registry, fixture modules, a claims manifest, an exact-arithmetic oracle and a static checker.

Implementing MeTTaDinC in Cetta: A Coding-Agent Specification and Work Plan (30 pages). An engineering document that hands the implementation to coding agents. It states the invariants an implementation must keep, the intermediate representation and backends, the default algorithms and their scope, certificate formats and their replay, the trusted boundary of the symbolic path, the extension points of Cetta, and eight milestones from strict result boundaries to the Hyperseed library, each with acceptance claims and a negative test suite that points to fixtures in the other documents.

MeTTaDinC in Practice: Four Worked Examples (79 pages). Four complete programs, each worked out from the problem statement to the expected result of every request: a belief memory under memory pressure, an access-control policy audited without enumeration, skill sharing among three robots with different camera frames, and market making across regime shifts. Each example gives the modules with the reasons for each declaration, the results with their witnesses and certificates, notes for implementers, and the registry additions it proposes. The bundles ship 628 proofs that a verified checker replays.

Omega Memory and Goals in MeTTaDinC: A Practical Implementation Guide (Part I, 66 pages). How Omega agents can give their memory and goal machinery a MeTTaDinC layer. It describes WMTM, the planned OCO/2 runtime and GOLEM-Iter's promotion gate, then works through eight patterns as finite models: open items and the goal engine, the context block, consolidation and anchor cues, task checkpoints, goal changes and their six kinds, conflicting claims, motivational state, and a stale-suggestion monitor. It ends with the integration: how results are

stored, how components consult them, how the gate consumes them, and the API an agent uses.

Omega Memory and Goals in MeTTaDinC, Part II: Live Memory through Summary Statistics (53 pages). Extends Part I from finite models to live stores of any size. It introduces summary statistics checked by solvers for every state of real-valued carriers, the lifting of item-level results to keyed stores, safety carried through simulations, anytime-valid estimates and monitors for what logs alone can show, with a proposition that keeps one-sided claims valid when rates vary, and an eight-step protocol for applying the method to one live store.

Omega Memory and Goals in MeTTaDinC, worked example: WMTM's Open-Item Store, from Journal to Gate (45 pages). Carries Part II's protocol out end to end on eight simulated weeks of WMTM's goal engine: declaring the store, designing and freezing the receipts, catching a parser regression with a changepoint monitor, reporting the claims at the promotion gate, and checking the next release, with its snooze action and three-day goal, before it ships.